

## Lab 10 – Intermediate Code Generation (Postfix and Prefix)

### Aim

To generate postfix and prefix forms of an arithmetic expression for intermediate code generation.

### Algorithm

#### Postfix

1. Start the program.
2. Read the infix expression.
3. Scan the expression from left to right.
4. If the symbol is an operand, add it to output.
5. If the symbol is (, push it onto the stack.
6. If the symbol is ), pop until ( is found.
7. If the symbol is an operator:
  1. Pop operators from stack while stack top has higher or equal precedence.
  2. Push current operator.
8. After scanning, pop all remaining operators.
9. The obtained expression is postfix.

#### Prefix

1. Reverse the infix expression.
2. Exchange ( and ).
3. Convert the modified expression to postfix.
4. Reverse the postfix expression.
5. The obtained expression is prefix.
6. Stop.

### C Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAX 100
```

```
char stack[MAX];
int top = -1;
```

```
void push(char c) {
    stack[++top] = c;
}
```

```
char pop() {
```

```

    if (top == -1)
        return '\0';
    return stack[top--];
}

char peek() {
    if (top == -1)
        return '\0';
    return stack[top];
}

int precedence(char c) {
    if (c == '^')
        return 3;
    if (c == '*' || c == '/')
        return 2;
    if (c == '+' || c == '-')
        return 1;
    return 0;
}

void reverse(char str[]) {
    int i, j;
    char temp;
    for (i = 0, j = strlen(str) - 1; i < j; i++, j--) {
        temp = str[i];
        str[i] = str[j];
        str[j] = temp;
    }
}

void infixToPostfix(char infix[], char postfix[]) {
    int i, j = 0;
    top = -1;

    for (i = 0; infix[i] != '\0'; i++) {
        char c = infix[i];

        if (isalnum(c)) {
            postfix[j++] = c;
        }
        else if (c == '(') {
            push(c);
        }
        else if (c == ')') {
            while (peek() != '(')
                postfix[j++] = pop();
            pop();
        }
        else {

```

```

        while (precedence(peek()) >= precedence(c))
            postfix[j++] = pop();
        push(c);
    }
}

while (top != -1)
    postfix[j++] = pop();

postfix[j] = '\0';
}

void infixToPrefix(char infix[], char prefix[]) {
    char temp[MAX];
    int i;

    strcpy(temp, infix);
    reverse(temp);

    for (i = 0; temp[i] != '\0'; i++) {
        if (temp[i] == '(')
            temp[i] = ')';
        else if (temp[i] == ')')
            temp[i] = '(';
    }

    infixToPostfix(temp, prefix);
    reverse(prefix);
}

int main() {
    char infix[MAX], postfix[MAX], prefix[MAX];

    printf("Enter infix expression: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);
    infixToPrefix(infix, prefix);

    printf("\nPostfix Expression: %s\n", postfix);
    printf("Prefix Expression : %s\n", prefix);

    return 0;
}

```

## Sample Input

A+B\*C

## Sample Output

Postfix Expression: ABC\*+  
Prefix Expression : +A\*BC

## Result

Thus the postfix and prefix forms of the given arithmetic expression were generated successfully.